Feature: The booking form

From build-plan: feature 12f

Size: heavy - two separate risks with real logic: the business's own questions (a list kept in order, added, changed and removed, with past answers kept), and what the booking form requires of a customer (email, phone, address), which the public booking API enforces and the booking window every client site installs must show the same way.

Branch: feature/12f-the-booking-form

Status: verified 2026-10-11: both steps built and reviewed, every finding fixed; backend 980, shared 177 and booking component 83 tests pass, the frontend builds and lints. Then a final review of the whole feature.

Goal

The owner sets the booking form on Settings: the business's own questions (up to 20, under the standard ones), started from a ready-made set for the trade (painting, clinic) or written from nothing; whether a customer must leave an email, a phone, or either one; for each service whether the address is asked and required, asked and optional, or not asked; and whether the form asks for the customer's yes to later texts. After this nobody changes `booking_question`, the contact rule, a service's address choice or `askLaterTextsYes` by hand or in a setup file to change any of it.

In scope

- A Booking form page, `/settings/booking-form`, in the section list after Days off.
- **The questions (12f.1)**: the business's questions in order, each with its words and whether it is required; add one, change one, move one up or down, remove one; at most 20. With no questions, "Start from the painting set" or "Start from the clinic set" fills the list, to change before saving. One save writes the whole list. A removed or changed question never changes a lead's saved answers: each answer already keeps its own copy of the question's words (`lead.answers`).
- **What the form requires (12f.2)**: the contact rule, one of "an email", "a phone" or "either one", per business; the address, per service, one of "asked and required", "asked, optional" or "not asked" (today's yes/no per service becomes the three-way choice); whether the form asks for the yes to later texts. The public booking API enforces the contact and address rules, and the booking window shows each field as required or optional the same way.
- Read-only for a role that may not change the business, as 12a.

Out of scope

- What the business later sends to a customer who said yes (offers, reminders to book again): a later feature; this only asks and records the yes, as feature 9 built.
- The texting number, the confirmation and reminder texts (12g). A business without text settings cannot ask for the yes yet; the page says so instead of offering the choice.
- Answer types other than text (a choice list, a date): every answer stays text (decision 5).
- Questions per service: the questions are the business's, asked for every service.
- Publishing the booking component to GitHub Packages (Frank's yes each time).

Build loop

Approved one step at a time (AGENTS.md). Heavy: `workflow.stepReview` is `every`, `checkpointCommits` enabled. After a step's plan has Frank's yes, nothing stops until the review: build, tests and checks, tick the box, write the build log entry and commit it to buildlogs, commit and push the step to `feature/12f-the-booking-form`, `/audit` scoped to the step, then the independent review. The planned stop is after the review. Blocking findings (P0/P1) are fixed by default; P2/P3 are recorded and carried. `/complete` makes the final pass and merges on Frank's yes.

Build steps

- ☑ **12f.1 The business's questions.** "On Summit I press Start from the painting set, change 'How many rooms?' to required, add 'Any pets?', move it up, and save: the booking preview asks the three in that order. Removing 'Any pets?' later leaves Maria's lead still showing her answer to it."
  - Backend: `GET /settings/booking-form` (any member) answers `{ canEdit, questions, presets }`; `PUT /settings/booking-form/questions` (`organization: ["update"]`) saves the whole list in order in one transaction under a lock on the business's questions: kept ids are updated, missing ones deleted, new ones added; an id of another business or unknown is `404`.
  - Shared: the save's Zod schema (1 to 200 characters, trimmed, at most 20, no two the same words). The two ready-made sets are data in `backend/lib/settings/booking-question-sets.ts` (amended in 12f.1: only the backend reads them, through the GET).
  - Frontend: `app/(dashboard)/settings/booking-form/page.tsx`, "Booking form" in the section list, `components/settings/booking-form-screen.tsx` and `booking-questions-card.tsx`. **Done when:** the route tests below pass, and by hand on Summit the scene above works in the booking preview at http://localhost:3400/admin/booking-preview/painting-dev; Summit's questions are put back after.
- ☑ **12f.2 What the form requires.** "Riverbend sets 'a phone' and its facial service's address to 'not asked': the booking preview asks for a phone, marks the email optional, and asks no address; a booking sent without a phone is refused. Summit sets its estimate's address to 'asked, optional': a booking without one is taken."

- Migration: `organization.contactRule` (`email` | `phone` | `either`, existing businesses `either`, today's rule); `booking_link.addressMode` (`required` | `optional` | `not_asked`) replaces `asksAddress` (true becomes `required`, false `not_asked`).
- Backend: the booking route checks the contact rule and the address mode; the public service detail answers both; `PUT /settings/booking-form/rules` saves the contact rule and `askLaterTextsYes` (only for a business with text settings, else `409 no_text_settings`); the service save takes `addressMode`. Client setup files and the seed say both.
- Frontend: the rules card on the Booking form page; the service form's address choice becomes three-way. Booking component: the contact fields and the address follow the rules. **Done when:** the booking route and booking component tests below pass, and by hand both scenes above work in the booking preview; the dev businesses are put back after.

Files / areas

- `frontend/app/(dashboard)/settings/booking-form/`, `frontend/components/settings/` (section list, the questions card, the rules card, `service-form.tsx`), `frontend/lib/api-client/settings/`
- `backend/lib/settings/` (find and save the booking form), `backend/routes/settings-routes.ts` and its test; `backend/lib/booking/` (book-time's contact and address checks, check-answers), `backend/routes/public-booking-links-routes.ts`, `public-bookings-routes.test.ts`
- `packages/shared/db/` (organization, booking_link), a migration, `packages/shared/zod-validation/` (the two saves, the service schema, create-booking), `packages/shared/client-setup/`, `packages/shared/scripts/seed-dev.ts`
- `packages/booking-component/booking-window/` (the form, its reading and its tests)

Data / contracts

All under the dashboard CORS, CSRF and no-store middleware and the `booking` module; reads for any member, writes need `requirePermissionMiddleware({ organization: ["update"] })`. The business always comes from the session; every query filters on it first.

`GET /settings/booking-form` `200 { canEdit, questions, presets, contactRule, askLaterTextsYes }`: `questions` `{ id, label, required }` in order; `presets` `{ trade: "painting" | "clinic", name, questions: { label, required }[] }`; `contactRule` and `askLaterTextsYes` from 12f.2 (`askLaterTextsYes` null when the business has no text settings).

`PUT /settings/booking-form/questions` `{ questions: { id?, label, required }[] }`, strict, 0 to 20, labels 1 to 200 characters trimmed, no two the same (ignoring case). Order is the list's order. `200 { questions }` as the GET's. `404 not_found` for an id not this business's.

`PUT /settings/booking-form/rules` (12f.2) `{ contactRule, askLaterTextsYes? }`, strict; `askLaterTextsYes` left out when the page did not show it (no text settings), and then left as it is. `200 {`

`contactRule, askLaterTextsYes }` as saved (amended at 12f.2: the card needs only these two); `409 no_text_settings` for a yes without text settings.

Booking (public): `customer.email` required when the rule is `email`, `customer.phone` when `phone`, one of them when `either` (today's rule); `location`, when the service's `addressMode` is `required`, refused empty and taken missing (left blank: the form never showed the box; amended after 12f.2's review), taken when `optional`, ignored (not kept) when `not_asked` (Frank, 2026-10-11, at 12f.2's plan: a form opened before the address was switched off still books). Errors keep the public route's shape, `400 { error: { code, message } }` with no `field` (amended at 12f.2: the booking window marks the field itself, with the same `missingContact`, before anything is sent). `answers` (amended after 12f.1's review) carries every question the form showed, `{ questionId, question, answer }`, the words as shown and a blank answer included: a required question shown and left blank is `400 "Answer: <words>"`; one the form never showed is left blank; an answer to a question removed since is kept under its shown words.

User text: question words and answers are rendered by React as text, never HTML.

Testing

Commands: `npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend` (local Postgres, migrated and seeded), `npm run test --workspace=@frankdmosquera/booking-component`, `npm run build` and `npm run lint` for the frontend.

One test per rule that would hurt if broken:

- 12f.1: a save keeps the order, changes and removes; another business's question id is refused and nothing is saved; a removed question leaves a lead's saved answer; a member without the permission cannot save.
- 12f.2: each contact rule refuses a booking without what it requires and takes one with it; each address mode refuses, takes or ignores the address as it says; the booking window marks the fields the same way; the yes to later texts cannot be turned on without text settings.

Notes for the AI

- 12a's patterns: client components through `dashboardApiClient`; each error under its field with `aria-describedby`; a refused save focuses the first bad field; saved and failed messages announced and cleared on the next edit. No TanStack Query (feature 14).
- Nothing is pre-filled where the business must choose (decision 30): the ready-made sets fill the list only when pressed; `contactRule` keeps today's rule for existing businesses.
- Comments beside the line they explain, one or two lines, no step or finding numbers.

Decided

- The ready-made sets' questions (Frank, 2026-10-11, at 12f.1's plan): the dev seed's own, painting "Interior or exterior?" (required) and "How many rooms?"; clinic "Any allergies or skin conditions?" (required) and "Is this your first visit?". More can be added any time.
- A booking sent from a form opened before the owner changed the questions is always taken (Frank, 2026-10-11, at 12f.1's review): answers keep the words the customer read, a newly required question they never saw stays blank, and a fresh form still requires it.
- The same for the address (Frank, 2026-10-11, at 12f.2's review): a form opened before a service's address was made required sends none, and its booking is taken with the address blank; a form that showed the box and left it empty is still refused.

Implementation walkthrough

Feature 12f was specced heavy with two steps, one per real risk, and kept them: the business's own questions (12f.1, `2739035`, review fixes `d4dc8fa`), and what the form requires of a customer (12f.2, `9c102b2`, review fixes `25cef02`). Each step had a fresh independent review; a final review of the whole feature at `7e13ff2` passed, closed F-384 to F-392, and left one P3 open (F-393). Frank decided three things along the way: a booking sent from a form opened before the owner changed the questions is always taken (F-384); an address sent for a service that does not ask for one is ignored, not refused (12f.2's plan); and a form opened before a service's address was made required is taken with the address blank (F-391).

the questions on Settings (`backend/lib/settings/`, `routes/settings-routes.ts`)

`find-booking-form.ts` answers the Booking form page in one read: the questions in order, the two ready-made sets from `booking-question-sets.ts` (the dev seed's own painting and clinic questions, only the backend reads them), the contact rule, and the yes to later texts, which is null when the business has no text settings. `save-booking-questions.ts` saves the whole list in one transaction under `lockBusinessResources` ("no key update", so bookings and leads for the business are never held up, F-386): every id sent must be this business's, missing ones are deleted, kept ones updated in place with their new position, new ones added. An unknown id is one 404 whose words tell the owner to reload (F-388). A lead keeps its own copy of each question's words, so nothing here touches past answers. `save-booking-form-rules.ts` saves the contact rule and, only when the page showed the choice, the yes to later texts (F-392); a yes without text settings is `409 no_text_settings` before anything is written.

the booking route (`backend/lib/booking/`, `routes/public-bookings-routes.ts`)

`check-answers.ts` takes every question the form showed, with the words it showed and a blank answer included (F-384). A required question the form showed and left blank is refused with its words; one the form never showed is left blank; an answer to a question removed since the form opened is kept under the

words shown, after the business's own; a reworded question keeps the words the customer read. `book-time.ts` reads the service's `addressMode` and the business's `contactRule` in the same query. For a customer's booking it runs `missingContact` (the one check the booking window runs too) and keeps the address only when the service asks for it; a required address left empty is refused by the shared schema, and a form that sent none never showed the box, so it books with the address blank (F-391). The owner's own bookings skip both, as before. The public service list answers each service's `addressMode` and the business's `contactRule`.

shared (`packages/shared/`)

Migration 0028 adds `organization.contactRule` (default `either`, every business's rule so far: Better Auth makes the row before any setup file runs) and `booking_link.addressMode`, filled from `asksAddress` (true became `required`, false `not_asked`) before it is made NOT NULL; 0029 drops `asksAddress`. They are two migrations so drizzle-kit would not ask whether the new column is a rename. `helpers/missing-contact.ts` is the contact check (F-390 moved it out of `zod-validation/`). The questions save schema and the setup file share one list rule, at most 20 and no two the same (F-389). The create-booking schema no longer insists on an email or a phone: the business's rule decides. Setup files say `contactRule` and each service's `addressMode`; the client setup sets the rule only while the business has no service yet and lists a different one as a difference after that. The seed and `agentsweb.ts` keep `either` and today's address choice.

frontend

`settings/booking-form` is the Booking form page, after Days off: `BookingFormRulesCard` (what a customer must leave, and the yes to later texts or a line saying texts are not set up yet) above `BookingQuestionsCard` (the list with Up, Down, Remove, "Must be answered", the two ready-made sets when the list is empty, and one Save). After a move or a remove focus stays on the row's own button or the nearest one, and a status line says where it went (F-385); each box is named for its question (F-387). The service form's address choice is three-way: "Asked and required: we go to them", "Asked, optional", "Not asked", unpicked for a new service.

the booking window (`packages/booking-component/`)

`read-booking-form.ts` sends every question shown with its words, runs the same `missingContact` before sending, and sends the address always when required (so an empty one is said under its box), only when written when optional, never when not asked. `booking-form-view.tsx` marks Email or Phone required by the rule, keeps "An email or a phone" under both only for either one, and shows the address required, unmarked or not at all.

tests

Route tests cover the questions save (order, change, remove, another business's id, a removed question leaving a lead's answer, permission, the lock not waiting on a lead being written), each stale-form path,

each contact rule, each address mode with a required address refused empty and taken missing, and the rules save with and without text settings. Unit tests cover `check-answers`, `missing-contact`, the setup list rule, and the window's form reader and view. Each new rule's test was seen failing against the old code. At the end: backend 980, shared 177, booking component 83, and the frontend builds and lints; the frontend has no test runner, so the pages were checked by hand on Summit and Summit was put back.

Carried

F-393 (P3, open): a booking's answers keep the words the request sends, so a stranger could add lines that read as the business's own questions. It follows from Frank's F-384 decision; marking answers whose words are not the live question's is his call.

Findings

12f/F-384 [P2] closed - A customer with the form open when the owner saves the questions gets a refusal they cannot fix

File: backend/lib/booking/check-answers.ts:21-29; backend/routes/public-bookings-routes.ts:116-129; backend/lib/settings/save-booking-questions.ts:48-70; packages/booking-component/api-client/send-booking.ts:69-71 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** Before this step a business's questions only ever grew (client setup adds them only when there are none); the Settings save now deletes and changes them, and the booking route checks answers against the live list at submit time. Three paths follow from the code: (1) an answer to a question removed meanwhile makes checkAnswers return `unknown_question`, and the route answers 400 "That is not one of this business's questions."; (2) a question newly made required, or newly added as required, that the open form never showed returns 400 "Answer: <question>" with no field to answer it in; (3) a question whose words were changed in place keeps its id, so the lead's snapshot stores the customer's answer under the new words, which they never read (the snapshot is meant to keep "the question's words as asked"). For (1) and (2) send-booking.ts maps any 400 to `refused` with `canRetry: false`, so the customer is stopped with a message that does not say to reload, and the booking is lost unless they reload and fill the form in again. **Suggested fix:** Let the booking route tell "the form changed" apart: ignore (or refuse with a distinct code such as `form_changed`) answers to ids no longer asked, and refuse a required question the request did not mention with the same code; have the booking window answer that code with "This form changed, please reload" (or reload the service's questions keeping the typed values). For (3), either send the words shown with each answer and keep those in the snapshot when the id still matches, or have the save give a question whose words changed a new id so (1) covers it. One route test per path. **Resolution:** Fixed 2026-10-11, Frank's call: the booking is always taken. The form sends every question it showed with its words, blank ones included; a required question it showed must be answered, one it never showed is left blank; an answer to a question removed meanwhile is kept under the words shown, after the business's own; the lead keeps the words the customer read. `unknown_question` is gone. Tests: check-answers.test.ts and

public-bookings-routes.test.ts (never showed, removed, reworded, shown and blank), each seen failing with the old rule. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): check-answers.ts no longer refuses an unknown or never-shown question, the route has no `unknown_question`, read-booking-form.ts sends every shown question with its words, and the four route tests (never showed, removed, reworded, shown and blank) pass. That the kept words now come from the request is recorded separately as F-393.

12f/F-385 [P2] closed - Moving a question down or removing one drops keyboard focus to the page

File: frontend/components/settings/booking-questions-card.tsx:43-47, 157-187 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** Rows are keyed by `row.key`, so on Down React keeps the next row in place and re-inserts the moved row's `li`; re-inserting a node that holds focus runs the browser's focus fixup and focus goes to `body`. Remove unmounts the focused button outright. Up from the second row leaves focus on a button that becomes `disabled`. A keyboard or screen-reader owner reordering the list has to Tab back from the top of the page after every Down and every Remove, and nothing announces the new position. The step was asked to keep Up/Down/Remove keyboard-reachable; they are reachable once, not usable in sequence. **Suggested fix:** After a move, focus the same control on the row's new index (or the opposite arrow when the pressed one is now disabled); after a remove, focus the next row's Remove (or the previous row's, or "+ Add a question" when the list empties). Refs keyed by row key and a small effect, as useFocusFirstInvalid does. Optionally announce "Moved to position N" in the existing `role="status"` region. **Resolution:** Fixed 2026-10-11: after Up/Down focus stays on the moved row's same arrow, or its other arrow at the end of the list; after Remove it goes to the next row's Remove, the previous one's, or + Add a question; a status line says "Moved to position N of M." or "Question removed.". Checked in the browser on /settings/booking-form (no frontend test runner yet). Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): re-read booking-questions-card.tsx: Up/Down set focus to the moved row's same arrow, or its other arrow at the end; Remove to the neighbour's Remove or + Add a question; the effect runs after the rows render, and the status line announces the move. Code read only (no frontend test runner).

12f/F-386 [P3] closed - The questions save locks the business row "for update", which also holds up every booking and lead written for that business

File: backend/lib/settings/save-booking-questions.ts:25-30; backend/lib/settings/lock-business-resources.ts:1-4, 20 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** Postgres takes `FOR KEY SHARE` on the referenced organization row whenever a row pointing at it is inserted (booking, lead, contact, booking_question), and `FOR UPDATE` conflicts with that, so while a questions save runs every public booking, lead and contact insert for the business waits on it. The project already has the helper for exactly this, lockBusinessResources, which uses `for("no key update")` and says why: it "still lets bookings and every other row that points at the business be written meanwhile". Two "no key update"

locks still exclude each other, so two question saves would still run one after the other. The wait is short (at most about 45 statements), hence P3, but it is drift from the established lock. **Suggested fix:** Take `for("no key update")` on the organization row (reuse lockBusinessResources, or a shared lockBusiness helper if the name should not say "resources"). **Resolution:** Fixed 2026-10-11: the save takes lockBusinessResources ("no key update"). Test: settings-routes.test.ts "a save does not wait for a lead being written for the business", which answers "waited" with "for update". Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): save-booking-questions.ts takes lockBusinessResources ("no key update"); settings-routes.test.ts "a save does not wait for a lead being written for the business" passes.

12f/F-387 [P3] closed - Every "Must be answered" box has the same accessible name

File: frontend/components/settings/booking-questions-card.tsx:145-154 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** With several questions a screen reader lists N checkboxes all called "Must be answered", nothing tying each to its question, while the text field beside it is "Question N" and the buttons are "Move question N up" and so on. Moving between form controls the owner cannot tell which question a box belongs to. **Suggested fix:** Give the box a row-specific name, for example an `aria-label` of "Question N must be answered", or `aria-describedby` pointing at the question's input, matching the buttons' naming. **Resolution:** Fixed 2026-10-11: each box is named "Question N must be answered". Checked in the browser. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): each box carries aria-label "Question N must be answered" in booking-questions-card.tsx.

12f/F-388 [P3] closed - After a save from another tab removed a question, every later save on this page is refused until a reload

File: frontend/components/settings/booking-questions-card.tsx:52-71; backend/routes/settings-routes.ts:353 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** The lock makes the last save win, except when the earlier one deleted a question the later page still lists: the later PUT carries that id and gets `404 "No such question here."`, shown as an error notice. The rows keep the stale id, so pressing Save again gives the same 404 every time; the message does not say the list changed or to reload. **Suggested fix:** Word the 404 for this case ("These questions changed since you opened the page. Reload to see them.") or have the page reload the list on a 404. **Resolution:** Fixed 2026-10-11: the 404 says "These questions changed since you opened the page. Reload to see them." Asserted in settings-routes.test.ts. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): the 404 in settings-routes.ts says the questions changed and to reload; asserted in settings-routes.test.ts, which passes.

12f/F-389 [P3] closed - A setup file can still give a business more than 20 questions or two the same, which the Settings page then refuses to save

File: packages/shared/zod-validation/admin-validation-schemas/client-setup-validation-schema.ts:17, 63; packages/shared/zod-validation/crm-validation-schemas/booking-questions-validation-schema.ts:29-45 **Found:** 2026-10-11 by independent review of step 12f.1 (scope: a315b25..2739035; lenses: quality, security, performance, tests) **Why it matters:** The question rules now live in two schemas that disagree: the setup file's `questions` takes any number of labels with no duplicate check, while the Settings save allows at most 20 and refuses two with the same words. A business set up with 21, or with a repeated question, opens Booking form with "+ Add a question" disabled and every save refused until it cuts the list, for a list it never wrote. The label rule (1 to 200, trimmed) is also restated rather than shared. **Suggested fix:** Export the question item schema and the list's max and duplicate refinement from booking-questions-validation-schema.ts and use them for `questions` in the setup schema. **Resolution:** Fixed 2026-10-11: the setup file's questions use setupQuestionsValidationSchema from booking-questions-validation-schema.ts, the same label rule, 20 cap and duplicate check as the Settings save. Test: run-client-setup.test.ts, seen failing with the old schema. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): client-setup-validation-schema.ts takes setupQuestionsValidationSchema, sharing the label rule, the 20 cap and the duplicate check; run-client-setup.test.ts passes.

12f/F-390 [P2] closed - The shared contact check `missingContact` lives inside `zod-validation/`, as a second export of a schema file

File: packages/shared/zod-validation/crm-validation-schemas/contact-rule-validation-schema.ts:14-25; packages/shared/zod-validation/index.ts:19 **Found:** 2026-10-11 by independent review of step 12f.2 (scope: d4dc8fa..9c102b2; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md says `packages/shared` is "kind, then area, then one file per export", and that "A helper both apps use lives in `helpers/` ... never inside `zod-validation/`, which holds only validation schemas". `missingContact` is exactly that helper (the booking route and the booking window both call it), and it is now the only function anywhere under `zod-validation/`, exported from a file named for `contactRuleValidationSchema`. Frank navigates by folder and file name, so the one rule both sides must share is filed where he would not look for it, and the published booking component imports it through the validation barrel. Moving it later means touching the component's imports again. **Suggested fix:** Move `missingContact` to `packages/shared/helpers/missing-contact.ts` (with its test beside it, as `textable-phone-number.ts` is), export it the way the other helpers are, and import it from there in book-time.ts and read-booking-form.ts. `contact-rule-validation-schema.ts` keeps only the schema and `ContactRuleType`. **Resolution:** Fixed 2026-10-11: `missingContact` moved to packages/shared/helpers/missing-contact.ts with its test beside it, exported as `@scheduleads-app/shared/missing-contact`; book-time.ts and read-booking-form.ts import it from there; contact-rule-validation-schema.ts keeps only the schema and `ContactRuleType`. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): missingContact is in packages/shared/helpers/missing-contact.ts with its test, exported as ./missing-contact; contact-rule-validation-schema.ts holds only the schema and type; the booking component's build and dist import check pass.

12f/F-391 [P3] closed - A customer with the form open when a service's address is switched on gets "Enter the address." with no box to enter it in

File: backend/lib/booking/book-time.ts:217; packages/booking-component/booking-window/screens/booking-form-view.tsx:269-270; packages/booking-component/booking-window/booking-form/read-booking-form.ts:54-55; packages/booking-component/api-client/send-booking.ts:69-70 **Found:** 2026-10-11 by independent review of step 12f.2 (scope: d4dc8fa..9c102b2; lenses: quality, security, performance, tests) **Why it matters:** The window reads the service's `addressMode` once, when it loads. If the owner changes it from "not asked" to "asked and required" while a customer has the form open, the window draws no Address box and sends no `location`; the route then refuses with 400 "Enter the address.", which send-booking.ts turns into `refused` with `canRetry: false`. The customer has nowhere to type an address and the message does not say to reload, so the booking is lost unless they reload and start over. This is path (2) of F-384 for the address: for questions Frank decided a form opened before a change is always taken, and at this step's plan he decided the opposite address direction (switched off) still books, but the switched-on direction still refuses. It predates this step (the yes/no `asksAddress` had the same gap) and the amended spec states "required" as refused, so this is a consistency question for Frank, not a defect against the spec. **Suggested fix:** Frank's call. To match the questions rule: the window already always sends `location` for a required service (an empty box is refused by the schema), so the route could take a booking for a required service whose request has no `location` key at all (a form that never showed the box), and still refuse an empty one; one route test for each. Or keep the refusal and word it "This form changed, please reload the page". **Resolution:** Fixed 2026-10-11, Frank's call (as F-384): a booking that sends no address for a service that requires one is taken with the address blank, since its form never showed the box; an empty address is still refused by the schema ("Enter the address."). The `address_needed` refusal is gone. Tests: public-bookings-routes.test.ts "refuses an empty address" and "takes a booking that sent no address", the second seen failing with the old check. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): book-time.ts has no address refusal, an empty location is refused by the schema's min(1), and both route tests ("refuses an empty address", "takes a booking that sent no address") pass.

12f/F-392 [P3] closed - The rules card saves "no" for the yes to later texts when it never showed that choice

File: frontend/components/settings/booking-form-rules-card.tsx:21, 28-31; backend/lib/settings/save-booking-form-rules.ts:22-28 **Found:** 2026-10-11 by independent review of step 12f.2 (scope: d4dc8fa..9c102b2; lenses: quality, security, performance, tests) **Why it matters:** When the page loads before the business has text settings, the card holds `askLaterTextsYes: null`, hides the choice, and on Save sends `askLaterTextsYes ?? false`. The route then runs `UPDATE text_settings SET askLaterTextsYes = false` for the business. Today no update happens because only the dev seed creates text settings, but once 12g lets the owner set up texts, a Booking form page left open (or loaded in another tab) before texts were set up will silently switch the box off the next time the owner saves only

the contact rule, a choice they were never shown. The card's one PUT carries two independent settings, and "not shown" is sent as "no". **Suggested fix:** Let the rules save leave the yes alone when the card did not show it: make `askLaterTextsYes` optional (or nullable meaning "unchanged") in bookingFormRulesValidationSchema, have the card send it only when it was shown, and skip the text_settings update when it is absent. One route test: a save without it leaves an existing `true` in place. **Resolution:** Fixed 2026-10-11: `askLaterTextsYes` is optional in the rules save and left as it is when absent; the card leaves it out when it did not show the choice. Test: settings-routes.test.ts "the rules save" now saves the rule from a page that did not show the choice and keeps the yes, seen failing with the old save. Closed 2026-10-11 by /audit independent (scope: current; lenses: quality, security, performance, tests): askLaterTextsYes is optional in bookingFormRulesValidationSchema, the card leaves it out when it showed no choice, saveBookingFormRules skips the text_settings update when it is absent, and the route test keeping an existing yes passes.

Independent review

Status: passed **Target commit:** 7e13ff259c8db48d495600db47e2d2ac1f317e8e **Base commit:** a315b25fb2977f2a23cdb4d721f99327d6ce707f **Base ref:** main **Spec hash:** 82020478fe0ee1269fd1a5b038d5caca1926ff8798b998ce1772d2a8d664b677 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-11T21:58:28Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-11T22:10:00Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (24 files, 177 tests)
- `npm run test --workspace=@frankdmosquera/booking-component` : pass (16 files, 83 tests; its pretest build, type check and dist import check included)
- `npm run test --workspace=backend` : pass (88 files, 980 tests, against the local migrated and seeded `scheduleads_dev`)
- `npm run lint --workspace=frontend` : pass
- `npx tsc --noEmit --incremental false -p tsconfig.json` (in `frontend` , after the component's prebuild wrote the API route types): pass
- `npx prettier --check` on every code file in the delta: pass
- `npm run format:check` : fail, only on `packages/shared/tsconfig.build.json` , unchanged since c95f44e and outside this delta
- `npm run build --workspace=frontend` : unavailable (not run, a dev server may hold 3400)

Evidence

- `HEAD` is 7e13ff259c8db48d495600db47e2d2ac1f317e8e; `git merge-base main HEAD` is a315b25fb2977f2a23cdb4d721f99327d6ce707f; sha256 of `blueprint/context/current-feature.md` matches the spec hash; the only paths differing from the target are `review.md` (this file) and the untracked `blueprint/ai-voice-proposal.md` (see Remaining risk).
- Reviewed the whole a315b25..7e13ff2 delta (5 commits, 82 files): the settings routes and `backend/lib/settings/` (find, save questions, save rules, the question sets, the lock), book-time's contact and address checks, check-answers, the public booking and service routes, the shared schemas, `missing-contact`, migrations 0028 and 0029, client setup and the seed, the frontend Booking form page, both cards, the service form, and the booking component's form reader and view, with their tests.
- Security: every settings query filters on the session's business first; both PUTs sit behind `requirePermissionMiddleware(CHANGE_BUSINESS)` and strict Zod schemas; question ids are regex-bound; the public answer list is capped at 20 with bounded words and the 64 KB body limit still covers the worst case; a booking with neither email nor phone passes the schema but every contact rule refuses it in book-time before anything is written, so the per-contact rate limit is never bypassed by a made booking; user text renders through React and React Email only.
- Performance: the questions save is at most about 45 statements under a "no key update" lock that leaves bookings free; the GET reads two small queries in parallel; nothing unbounded.
- Tests: each contact rule, each address mode, the stale-form paths for questions and the address, the rules save without text settings, the permission refusals, another business's question id and the lock are all covered by passing route tests; the window's phone rule and optional address by component tests. No skipped or focused tests found in the delta.
- F-384 to F-392 re-examined against the new code and closed; F-310 noted still open.

Findings

- F-393 [P3] open: a booking's answers keep the question words the request sends, so a stranger can add lines that read as the business's own questions (a trade-off of Frank's decided behaviour)
- F-384, F-385, F-386, F-387, F-388, F-389, F-390, F-391, F-392 closed
- No P0 or P1 finding is open or fixed

Remaining risk

- `npm run build --workspace=frontend` was not run (a dev server may be running on 3400); the frontend's type check and lint ran instead.
- `npm run format:check` fails on `packages/shared/tsconfig.build.json` (CRLF, outside this delta); every file in the delta passes Prettier.
- The frontend has no test runner: the Booking form page's focus handling, read-only view and both cards were reviewed by reading the code, not in a browser; `/check` was not run (not required).
- Migrations 0028 and 0029 were exercised only on the local dev database; no Railway database was touched, and migration snapshot drift was not checked with `db:generate` (it would write files).

- No dependency or vulnerability scanner is declared; none was run.
- The untracked `blueprint/ai-voice-proposal.md` is the owner's private draft, unrelated to the feature and never committed; it differs from the target but was not reviewed as code.